2.4 LINUX COMMANDS

Go through running notes of week 2 also best content there

🔥 **1. pwd**

🟢 **Simple**

Prints your current directory path.

pwd

🔵 **Deep**

- Stands for: **Print Working Directory**
- It tells you where you are in the filesystem

Example:

/home/indra/projects

👉 Think of it like your **current location in a map**

🌍 **Real Use**

- Avoid confusion when navigating
- Debug scripts

🔥 **2. whoami**

🟢 **Simple**

Shows the current logged-in user.

whoami

🔵 **Deep**

- Reads your current user ID from system
- Maps it to username

Example:

indra

🌍 **Real Use**

- Check permissions
- Debug “permission denied” errors

🔥 **3. clear**

🟢 **Simple**

Clears the terminal screen.

clear

🔵 **Deep**

- Doesn’t delete anything
- Just resets terminal display

👉 Old commands still exist in memory (scroll up)

🔥 **4. cd (MOST IMPORTANT)**

🟢 **Simple**

Change directory (move between folders)

cd folder_name

🔵 **Deep**

- Updates your current working directory
- Affects all future commands

🔥 **5. cd / (Root Directory)**

cd /

🟢 **Simple**

Moves to the root directory

🔵 **Deep**

Root / is:

👉 The **top-level directory of Linux**

Everything starts from here:

```
/
 ├── home
 ├── var
 ├── etc
```

🔥 **6. ls and ls -l**

🟢 **Simple**

ls

Shows file names

ls -l

Shows detailed info

🔵 **Deep**

ls -l shows:

-rw-r--r-- 1 user user 1024 Apr 16 file.txt

Breakdown:

- Permissions
- Owner
- Size
- Date

🔥 **7. cd ~ and Directories**

🟢 **Simple**

cd ~

Moves to your **home directory**

🧠 **What is Home Directory?**

Example:

/home/indra

👉 Personal workspace of user

🧠 **What is Root Directory?**

/

👉 Entire system's starting point

⚠️ **Don't Confuse**

Term	Meaning
/	Root (top of system)
~	Your home folder

🔥 **8. cd .. and cd ../..**

🟢 **Simple**

cd ..

Move one level up

cd ../..

Move two levels up

🔵 **Example**

If you are in:

/home/indra/projects/sql

Run:

cd ..

Now:

/home/indra/projects

Run:

cd ../..

Now:

/home

🔥 9. cd . (Important but misunderstood)

🟢 Simple

cd .

Does nothing (stay in same directory)

🔵 Deep

. means:
👉 Current directory

Used in:

- Scripts
- Commands like cp . file

🔥 All cd Commands Summary

Command	Meaning
cd folder	Go into folder
cd ..	Go one level up
cd ../..	Go two levels up
cd /	Go to root directory
cd ~	Go to home directory
cd .	Stay in same directory

🧠 Real Engineer Thinking

Don't just type cd.

Ask:
👉 "Where am I now?" → pwd
👉 "Where do I want to go?"
👉 "Relative or absolute path?"

🔥 **Command: ls (List Directory Contents)**

---

1. 🟢 Simple Explanation

ls shows files and folders in a directory.

ls

---

2. 🔵 Deep Explanation (What actually happens)

When you run:

ls

Linux does:

1. Opens the directory (using system call like opendir)
2. Reads entries (readdir)
3. Gets metadata (inode info if needed)
4. Prints names to terminal

🧠 **Key Internal Concept**

A directory is NOT a folder visually.

👉 It is a **table of filenames → inode numbers**

ls reads that table.

🔍 **Important Insight**

- ls without flags → only names (fast)
- ls -l → fetches extra metadata (slower)

---

3. 🌍 Real-World Use Cases

👷 **DevOps**

Check logs:

ls /var/log

Check recent files:

ls -lt

Debug permission issue:

ls -l

Check hidden config files:

ls -a

---

4. 🔧 Important Flags (Must Know)

🔹 **-l (long listing)**

ls -l

Output:

-rw-r--r-- 1 user user 1024 Apr 16 file.txt

Breakdown:

- -rw-r--r-- → permissions
- user → owner
- 1024 → size
- Apr 16 → last modified

🔹 **-a (all files, including hidden)**

ls -a

Shows:

. .. .bashrc

🔹 **-h (human-readable sizes)**

ls -lh

Instead of bytes:

1.2K 5M 2G

🔹 **-t (sort by time)**

ls -lt

Newest files first.

🔹 **-r (reverse order)**

ls -ltr

Oldest first.

🔹 **-R (recursive)**

ls -R

Lists everything inside subfolders.

🔹 **-d (directory itself, not contents)**

ls -ld folder_name

---

5. ⚠️ Common Mistakes

❌ **Mistake 1: Not seeing hidden files**

ls

👉 Missing .env, .git

Use:

ls -a

❌ **Mistake 2: Ignoring permissions**

ls

👉 You miss why file isn't accessible

Use:

ls -l

❌ **Mistake 3: Running dangerous recursive command**

```
ls -R /
```

❌ Can freeze terminal (huge output)

---

6. ⚡ Performance Insight

- ls → fast (names only)
- ls -l → slower (needs metadata)
- ls -R → very slow on large directories

👉 In production, always be careful with recursive commands.

---

7. 🔄 Related Commands (Very Important)

🔹 **tree (visual structure)**

```
tree
```

Output:

```
project/
 ├── file1
 ├── file2
```

👉 Better for understanding folder hierarchy

🔹 **find (search files deeply)**

```
find . -name "*.txt"
```

👉 Searches recursively

🔹 **stat (detailed file info)**

```
stat file.txt
```

👉 Shows full metadata

🔹 **du (disk usage)**

```
du -h
```

👉 Shows folder sizes

🔹 **file (file type)**

file file.txt

👉 Tells what kind of file it is

---

8. 🧠 Real Engineer Thinking

Don't just run ls.

Ask:

- "Do I need file names?" → ls
- "Need details?" → ls -l
- "Need hidden files?" → ls -a
- "Need latest files?" → ls -lt